Lab Sheet 02

Title: Designing use case and class diagrams

Aims: To design use case diagram and class diagram in BOUML tool

Task 01: Design class diagram for this scenario

Imagine a Library Management System designed to handle the operations of a public library. The system includes classes for Library, Book, and Patron.

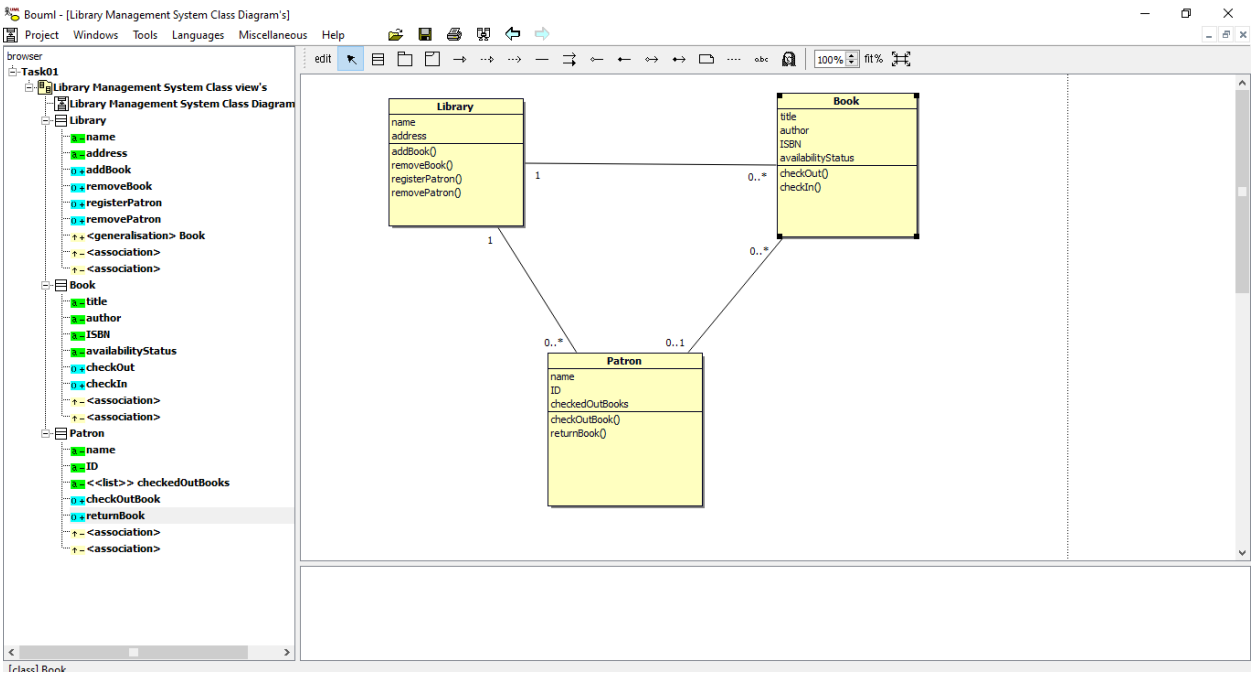
- **Library**: Represents a physical library facility with attributes such as name and address. It manages a collection of books and registered patrons. The library allows operations such as adding and removing books from its inventory, registering new patrons, and removing existing ones.
    - **Attributes**:
        - name: String
        - address: String
    - **Operations**:
        - addBook()
        - removeBook()
        - registerPatron()
        - removePatron()

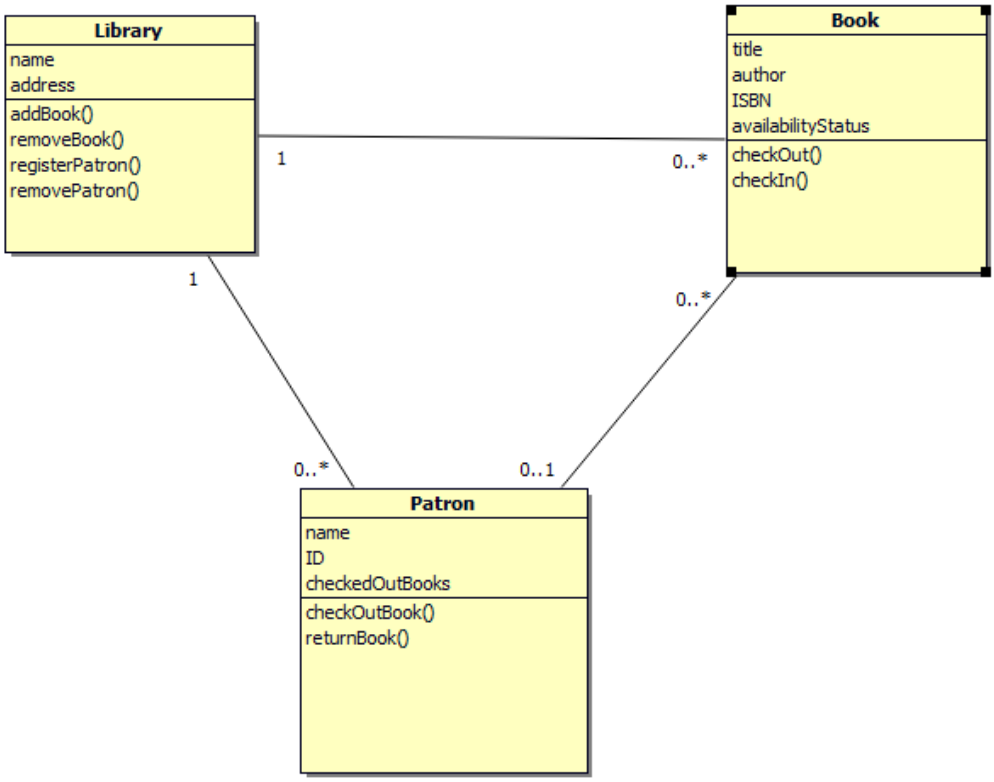
- **Book**: Represents individual books available in the library. Each book has attributes including title, author, ISBN, and availability status (whether it's currently checked out or available). Books can be checked out by patrons using methods provided by the library, such as checkOut() and checkIn().
    - **Attributes:**
        - title: String
        - author: String
        - ISBN: String
        - availabilityStatus: Boolean

- **Operations:**
    - checkOut()
    - checkIn()

- **Patron**: Represents individuals registered with the library. Each patron has attributes like name, ID, and a list of books currently checked out. Patrons can perform actions like checking out books (checkOutBook()) and returning books (returnBook()).
    - Attributes:
        - name: String
        - ID: String
        - checkedOutBooks: List<Book>
    - Operations:
        - checkOutBook()
        - returnBook()

The interactions between these classes are essential for managing the library's inventory and patron activities. The Library class interacts with Book and Patron instances to manage book inventory and facilitate patron services. Patrons interact with Book instances to borrow and return books, while Book instances maintain information about their availability and association with patrons.





Task 02: Design use case diagram for this scenario

The Online Banking System enables customers to perform various banking activities online, such as managing their accounts, transferring money, paying bills, and viewing transaction history.

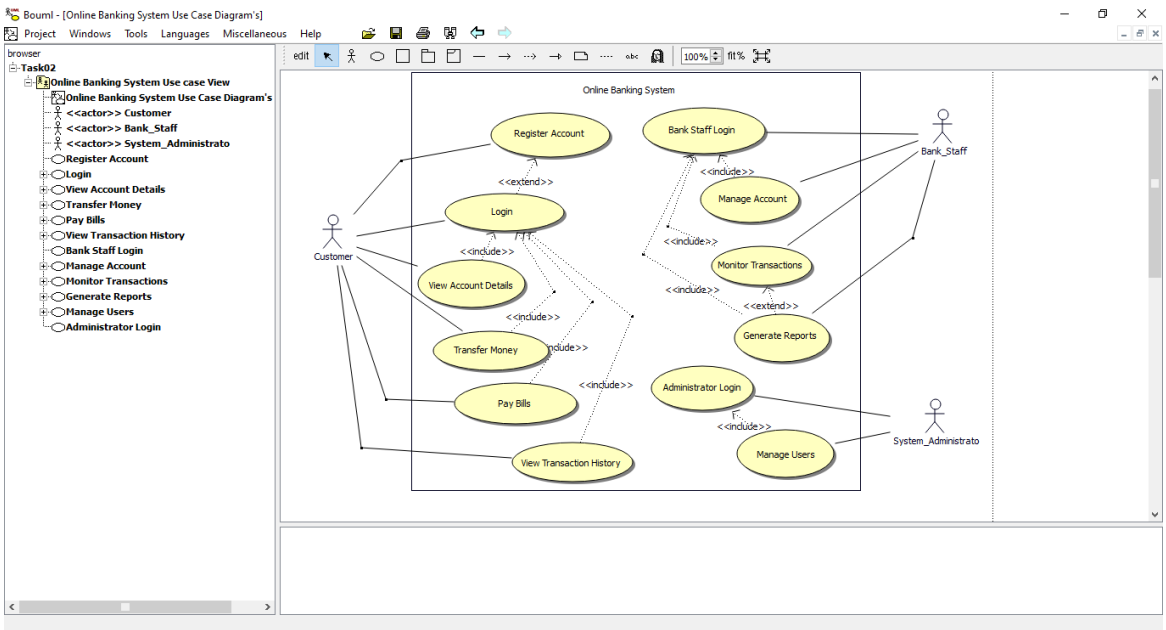
Actors:

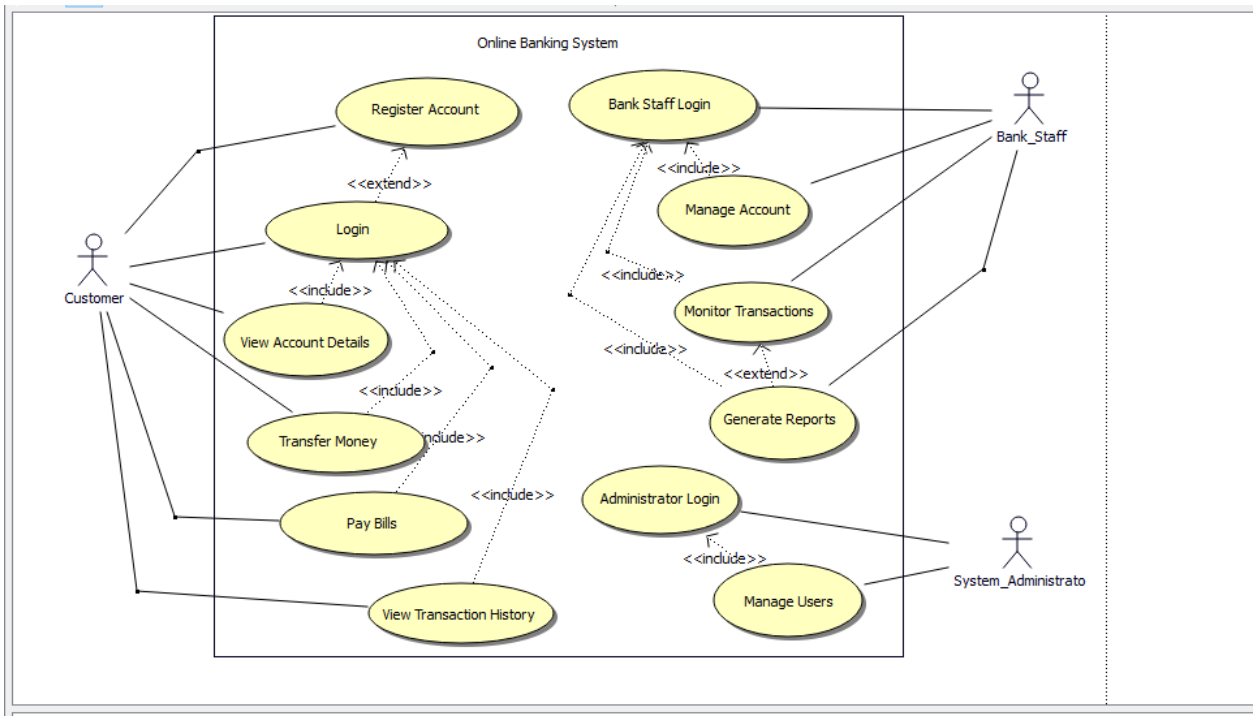
1. **Customer**: A user who has an account with the bank.
2. **Bank Staff**: Employees who manage customer accounts and transactions.
3. **System Administrator**: Manages system settings and user access.

Use Cases:

1. **Register Account**: A customer registers for an online banking account.
2. **Login**: A customer logs into their online banking account.
3. **View Account Details**: A customer views their account balance and details.
4. **Transfer Money**: A customer transfers money between accounts or to another customer.
5. **Pay Bills**: A customer pays their utility or other bills.
6. **View Transaction History**: A customer views the history of their transactions.

7. **Manage Account**: Bank staff manage customer accounts (e.g., update information).
8. **Monitor Transactions**: Bank staff monitor transactions for suspicious activity.
9. **Manage Users**: System administrator manages user access and permissions.
10. **Generate Reports**: Bank staff generate financial reports.





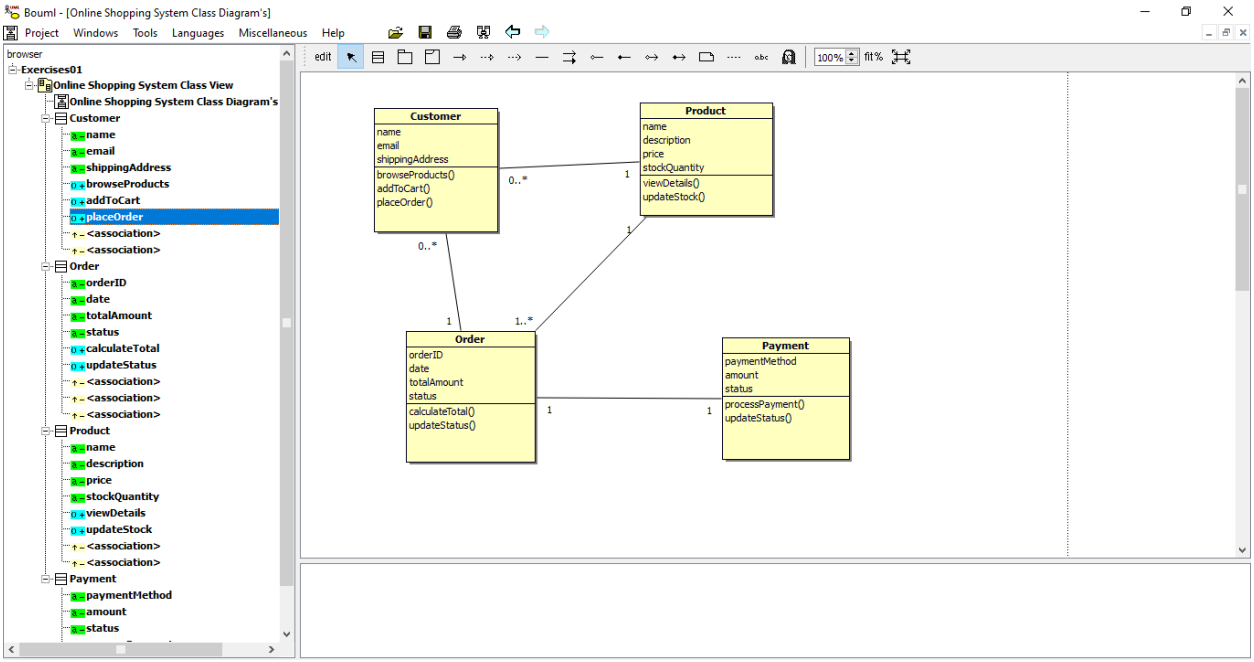
Exercises:

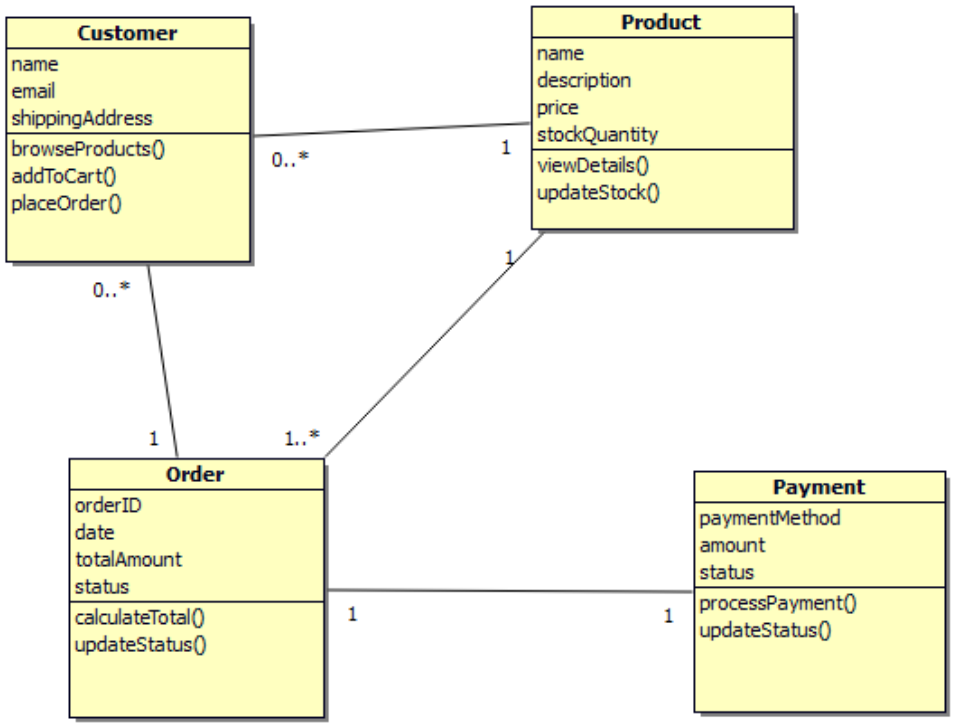
01) Imagine an Online Shopping System designed to facilitate e-commerce transactions. The system includes classes for **Customer**, **Order**, **Product**, and **Payment**.

- **Customer**: Represents a user registered with the online shopping platform. Each customer has attributes such as name, email, and shipping address. Customers can browse products, add items to their shopping cart, and place orders.
- **Order**: Represents a purchase transaction made by a customer. Each order has attributes like order ID, date, total amount, and status (e.g., pending, shipped). Orders contain one or more products that the customer has selected for purchase.
- **Product**: Represents individual items available for sale on the platform. Each product has attributes such as name, description, price, and stock quantity. Customers can view product details, add items to their cart, and purchase them.
- **Payment**: Represents a payment transaction associated with an order. Payment details include payment method (credit card, PayPal, etc.), amount, and status (e.g., successful, pending).

The interactions between these classes enable seamless e-commerce operations. Customers browse products, add items to their shopping cart, and proceed to checkout to create an order. Orders contain products selected by the customer, and each order triggers a payment transaction upon completion.

- **Customer** interacts with **Product** to view details and add items to their shopping cart.
- **Order** interacts with **Customer** to associate the order with the customer who placed it.
- **Payment** interacts with **Order** to finalize the transaction and update payment status.
- **Product** interacts with **Order** to track inventory changes when products are purchased.





02) The Hotel Reservation System allows guests to book rooms, manage their reservations, and check availability. Hotel staff can manage bookings, check guests in and out, and handle payments.

Administrators can manage the system settings and user accounts.

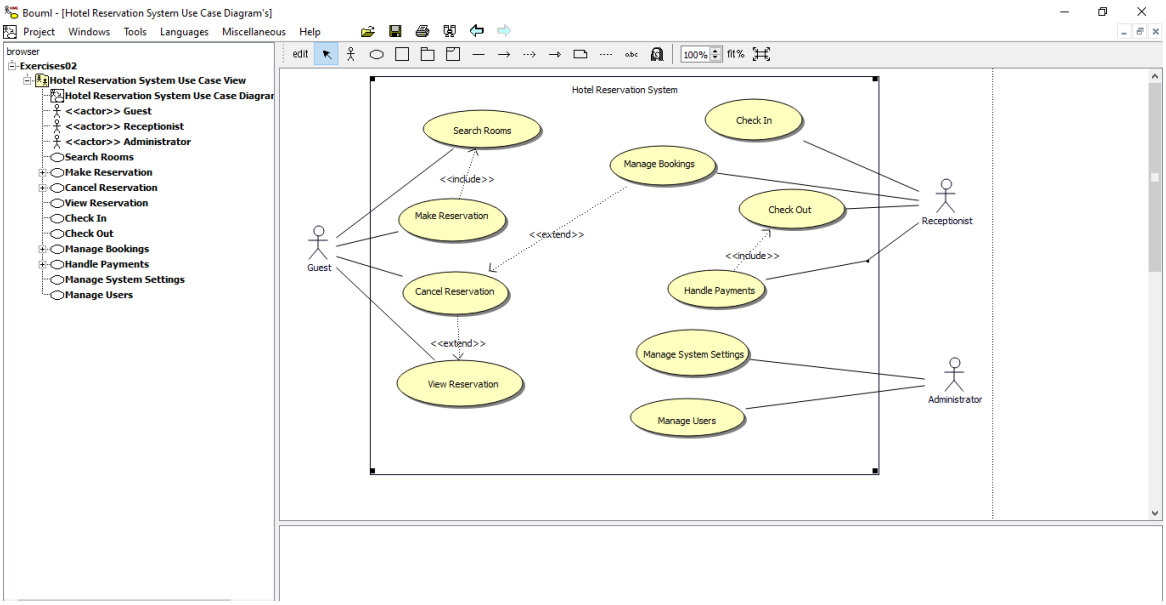
Actors:

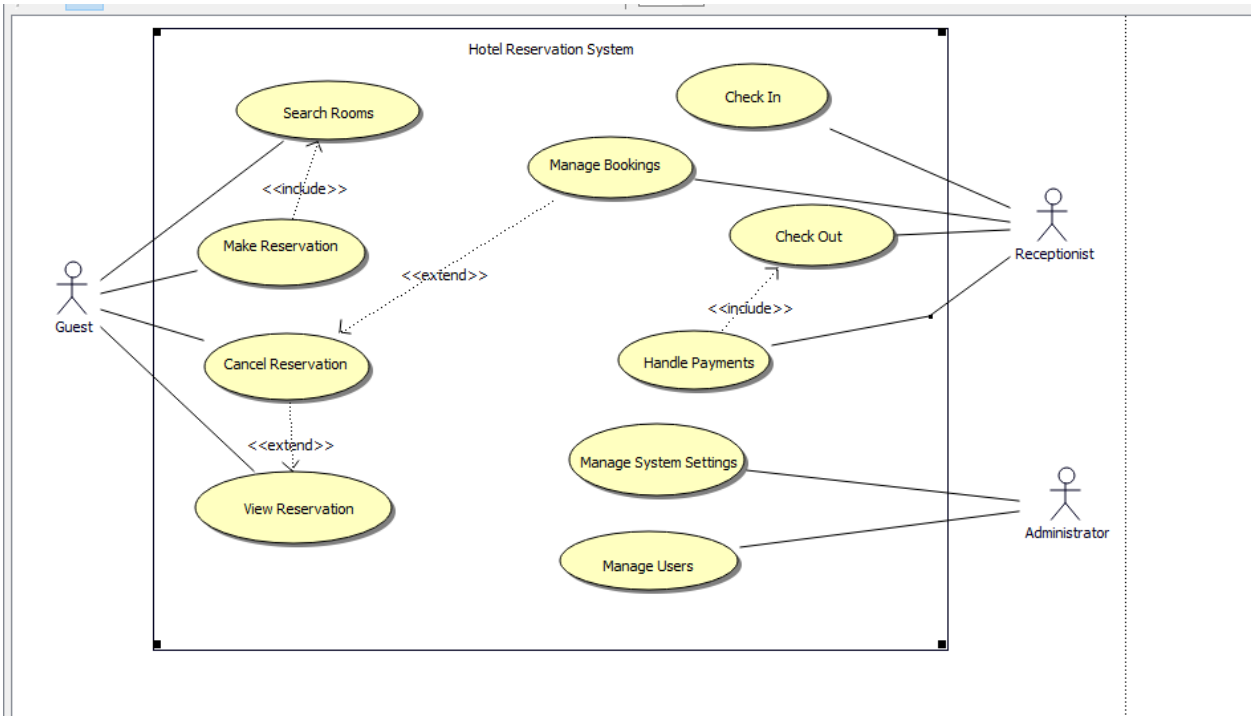
1. **Guest**: A user who wants to book a room at the hotel.
2. **Receptionist**: Hotel staff who manage bookings and guest services.
3. **Administrator**: Manages the hotel reservation system settings and user accounts.

Use Cases:

1. **Search Rooms**: A guest searches for available rooms based on dates and preferences.
2. **Make Reservation**: A guest books a room.
3. **Cancel Reservation**: A guest cancels a previously made reservation.
4. **View Reservation**: A guest views details of their reservations.
5. **Check In**: A receptionist checks in a guest upon arrival.
6. **Check Out**: A receptionist checks out a guest upon departure.

7. **Manage Bookings**: A receptionist manages room bookings (e.g., modify, cancel).
8. **Handle Payments**: A receptionist processes payments for room bookings and other services.
9. **Manage System Settings**: An administrator configures system settings.
10. **Manage Users**: An administrator manages user accounts and permissions.





03) Car Rental System

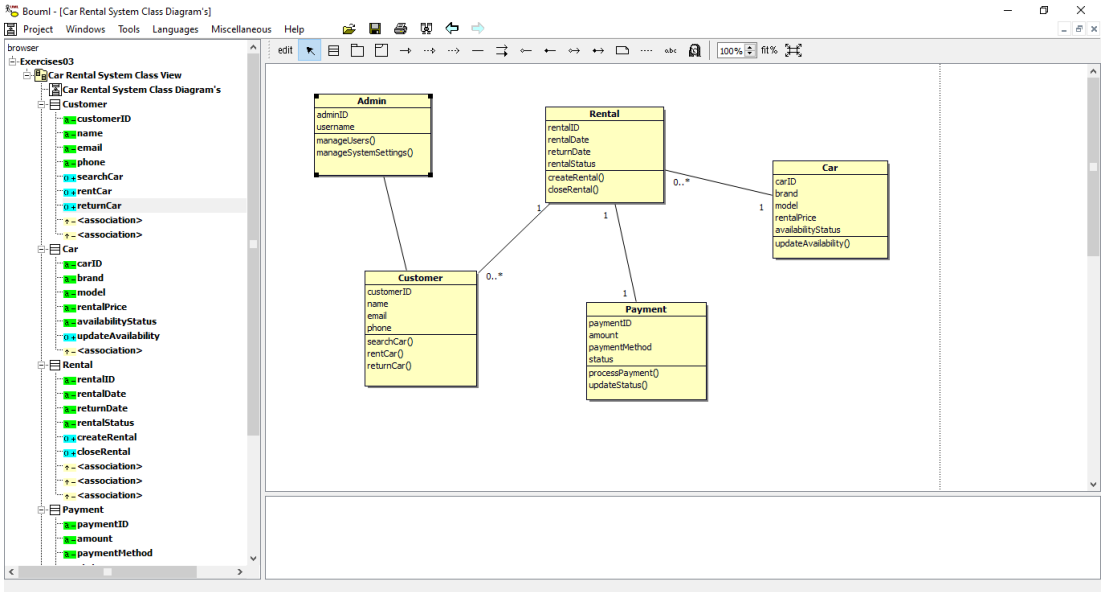
A **Car Rental System** allows customers to rent cars from a rental company. Customers can search for available cars, book rentals, and make payments online. Each rental transaction is recorded with details such as rental duration and payment status. The system also includes an administrator who manages the system settings and user accounts.

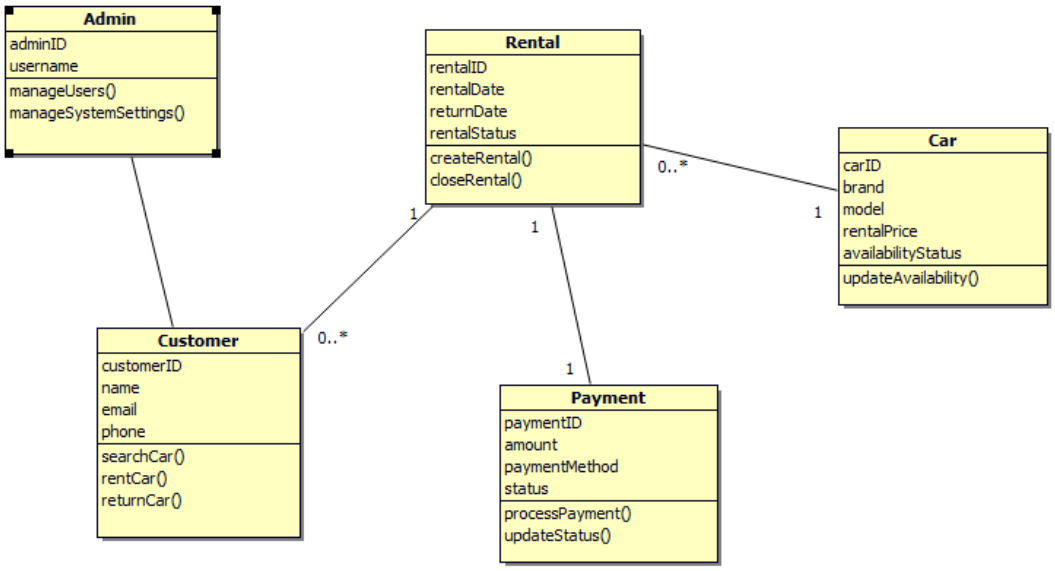
Classes and Relationships:

1. **Customer**: Represents users who rent cars. A customer can **search for cars, rent a car, and return a car**.
2. **Car**: Represents the vehicles available for rent. Each car has details such as **model, brand, rental price, and availability status**.
3. **Rental**: Represents a car rental transaction. It contains details such as **rental date, return date, and rental status**.
4. **Payment**: Represents payments made for rentals. A payment is linked to a **rental transaction** and has details like **amount, payment method, and status**.
5. **Admin**: Manages the system by **handling user accounts and configuring settings**.

Relationships:

- A **Customer** can have multiple **Rentals** (1-to-Many).
- A **Car** can be rented multiple times, but only **one active rental per car at a time** (1-toMany).
- Each **Rental** has exactly **one Payment** (1-to-1).
- An **Admin** manages the **system settings and user accounts** (Generalization).




Admin
adminID
username
manageUsers()
manageSystemSettings()
Rental
rentalID
rentalDate
returnDate
rentalStatus
createRental()
closeRental()
Car
carID
brand
model
rentalPrice
availabilityStatus
updateAvailability()
Customer
customerID
name
email
phone
searchCar()
rentCar()
returnCar()
Payment
paymentID
amount
paymentMethod
status
processPayment()
updateStatus()
0..*
1
1
1
0..*
1